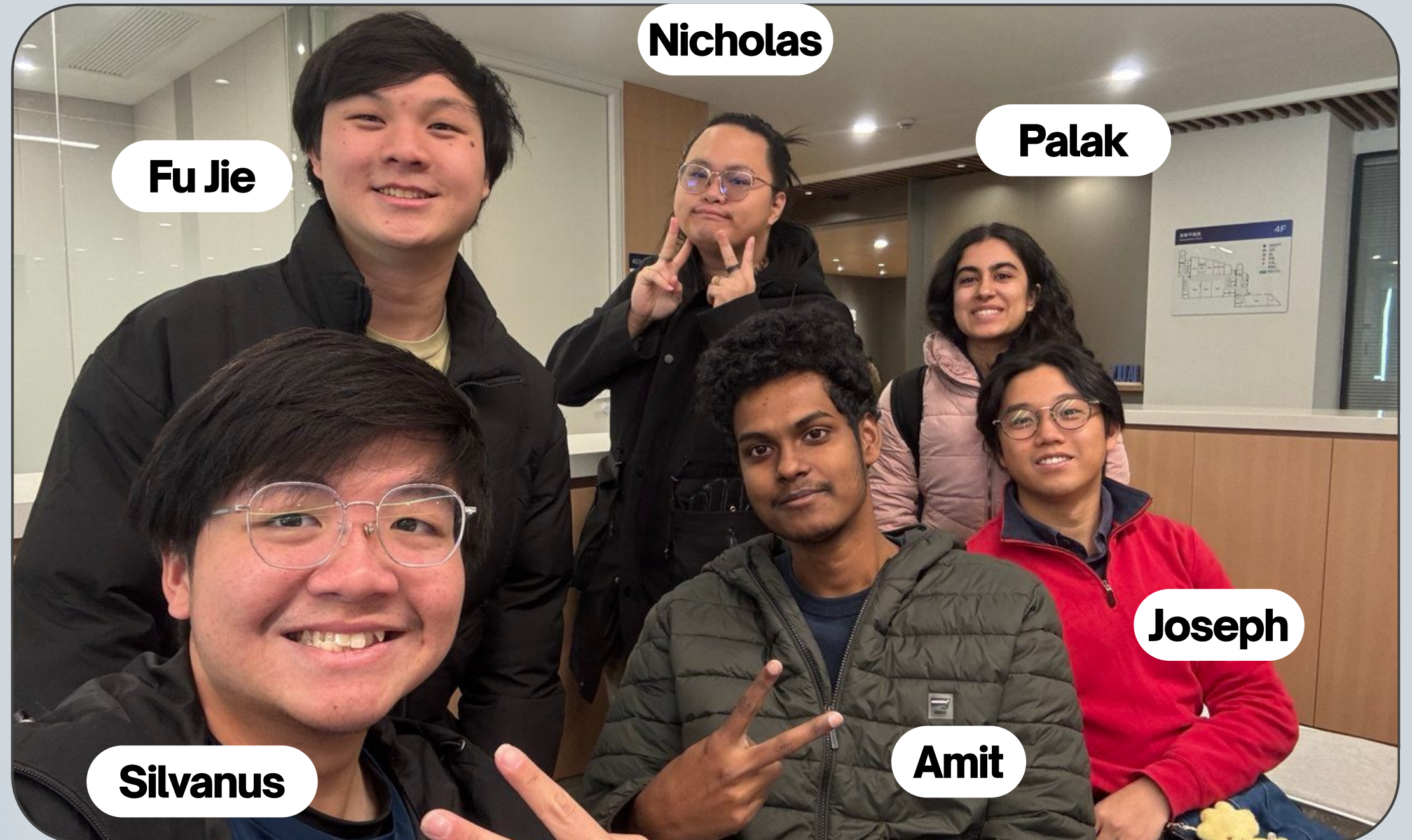


MIXIN.

Internship Report

Meet Our Team



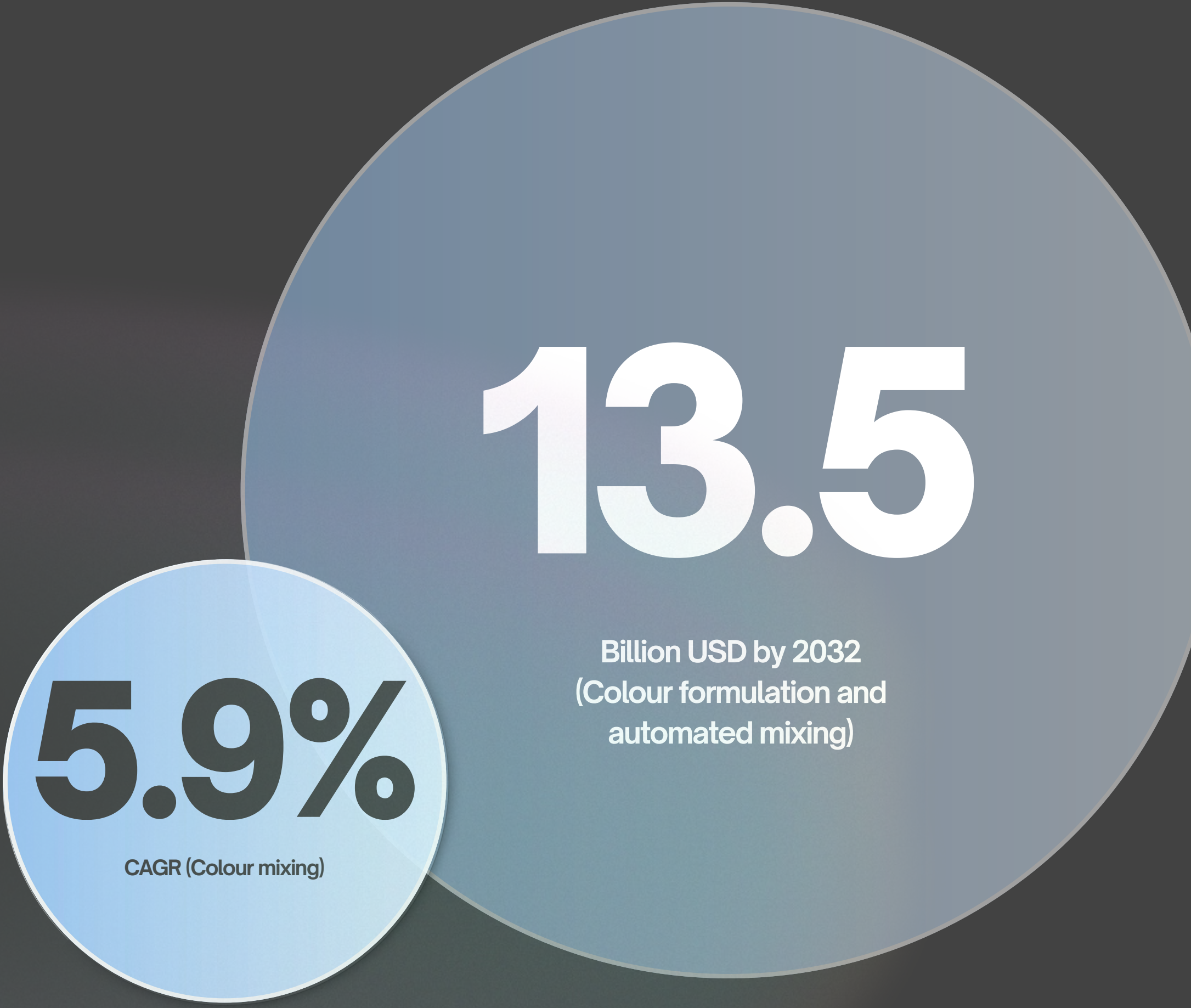
Problem Statement

How can we make a low cost workflow for hobbyist and designers to prototype?

Size of Market

The market for precision color tools is expanding within the massive arts and crafts materials industry, which exceeds \$23 billion. A key driver is the **paint-mixing sub-segment, which is on a trajectory to reach approximately \$13 billion by 2032.**

Simultaneously, the integration of physical objects into digital workflows is fueling a parallel opportunity. This spans the \$6–7.5 billion 3D scanning hardware market and the **\$8–16 billion 3D modeling and software segment, projected for 2030.**



5.9%

CAGR (Colour mixing)

13.5

**Billion USD by 2032
(Colour formulation and
automated mixing)**

Our Solution

COLOUR MIXING DEVICE

We designed a 3D-printed automated colour mixing device that delivers precise and repeatable colour outputs. The system uses stepper motors, gears, syringes, and a linear actuator for controlled dispensing and mixing. An ESP32 microcontroller with an LCD interface enables user input and automated operation, reducing manual effort and material waste.

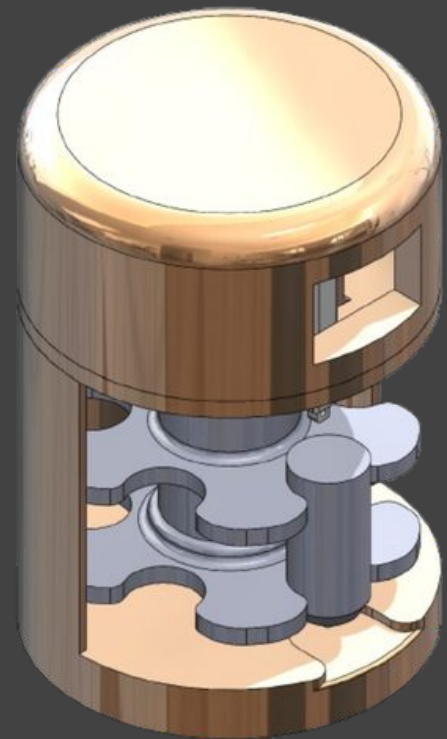


IMAGE TO STL BLENDER PLUGIN

We developed a web-based application and Blender plugin that converts images into editable Blender-compatible files. The web UI processes the input, while the plugin enables seamless import and editing within Blender. This streamlines the workflow from image to 3D model and reduces reliance on complex toolchains.

Iterations

Prototype 1



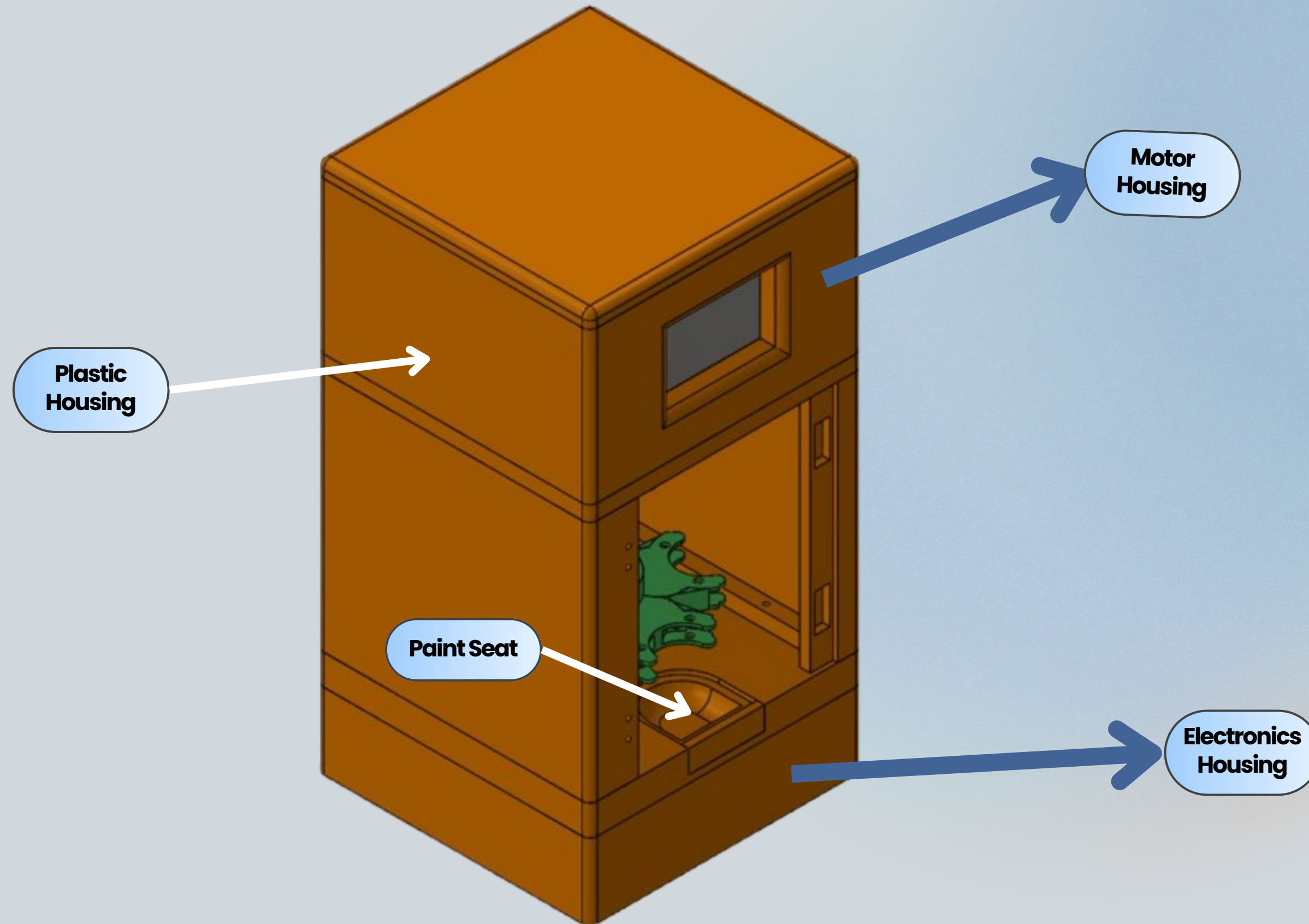
The first design featured a cylindrical plastic housing with a revolving door to enclose the internal components. It housed the syringe claw, linear actuator, stepper motors, and gears, providing functional operation but had limited usability and access.

Prototype 2



Based on company requirements for improved usability and manufacturability, the design was revised to a rectangular plastic housing with magnetic doors. Internally, it retains the same syringe claw mechanism, linear actuator, stepper motors, and gear assemblies for automated colour mixing.

Paint-Mixer



Paint-Mixer

**Plastic
Housing**

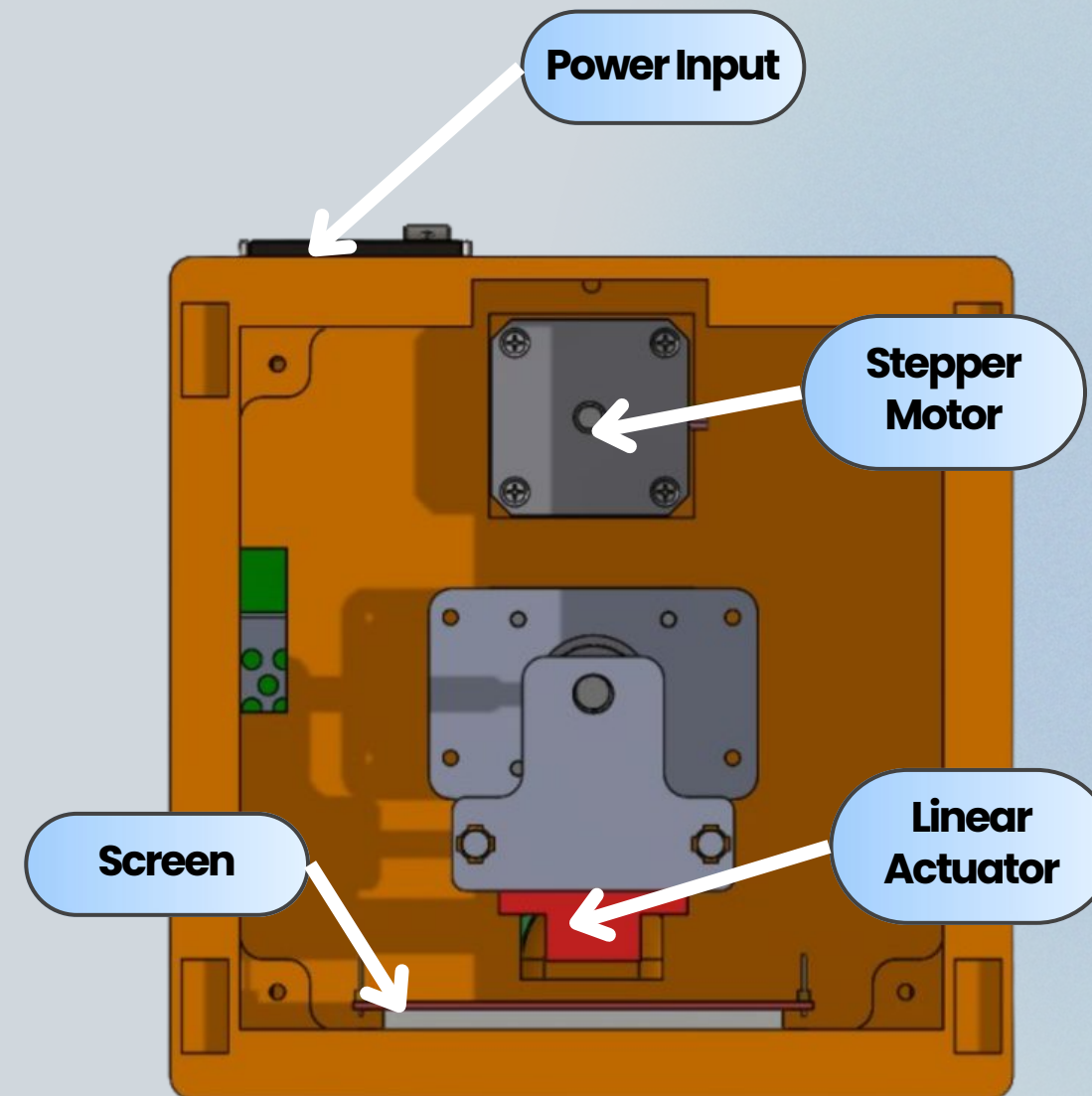


Material Selection

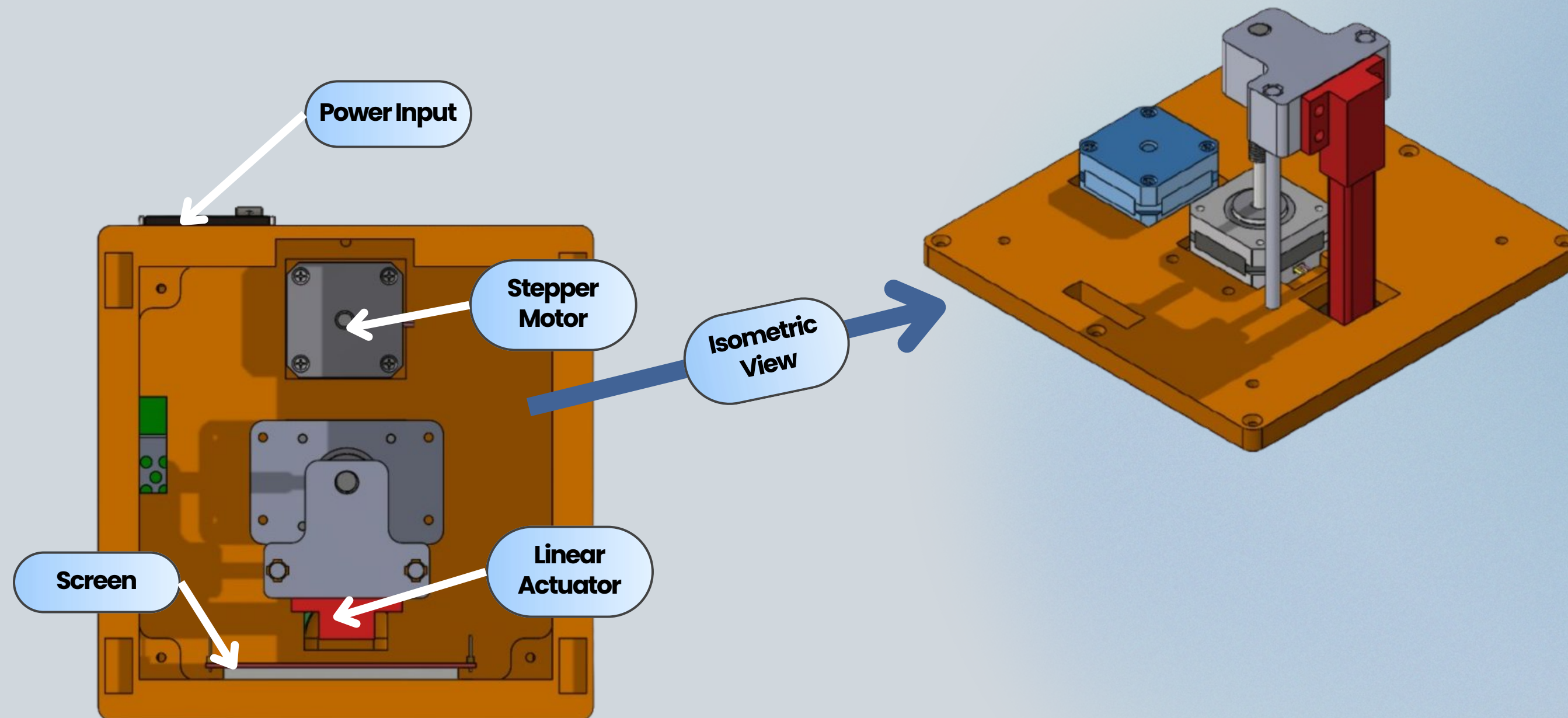
Material	Melting Temp (°C)	Glass Transition (°C)	Strength (MPa)	Stiffness (GPa)	Heat Resistance (°C)	Water Absorption (%)
PLA	180	60	60	3.8	60	0.65
PETG	240	80	55	2.1	80	0.3
PLA CF	210	60	80	7	80	0.5
PETG CF	250	80	75	5.2	90	0.3
ABS	240	105	50	2.4	100	0.4
ABS Fire Retardant	250	105	45	2.3	105	0.4
FR-ABS	250	105	45	2.3	105	0.4
Polycarbonate (PC)	290	145	70	2.3	145	0.3
PC-ABS	260	125	60	2.2	120	0.3
FR Nylon (PA)	240	60	75	2	100	1.5
PEI (Ultem)	360	215	100	3.4	190	0.3
PPS	285	90	100	3.5	220	0.1
PVDF	175	40	40	1.5	140	0.08

Paint-Mixer

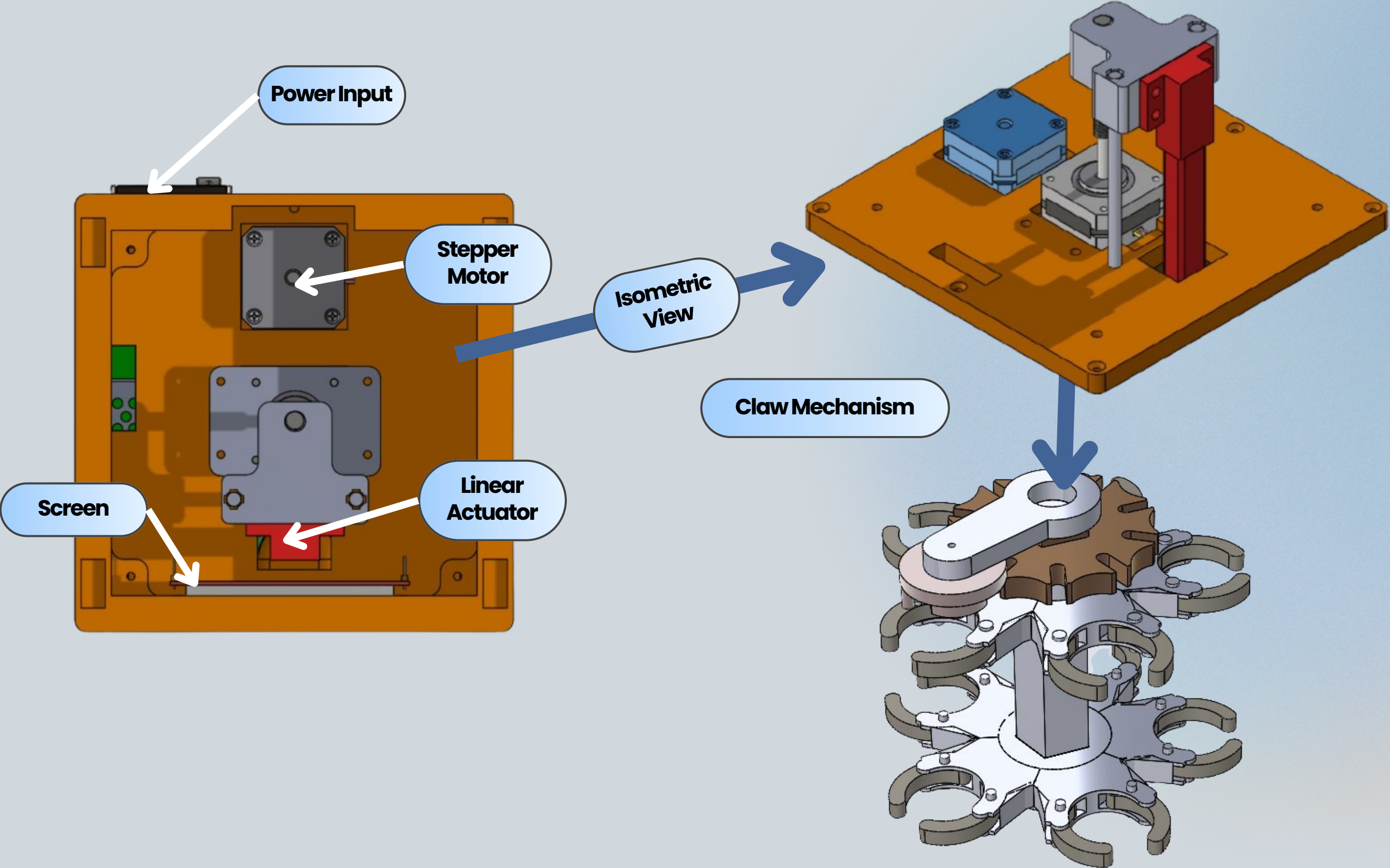
Motor Housing



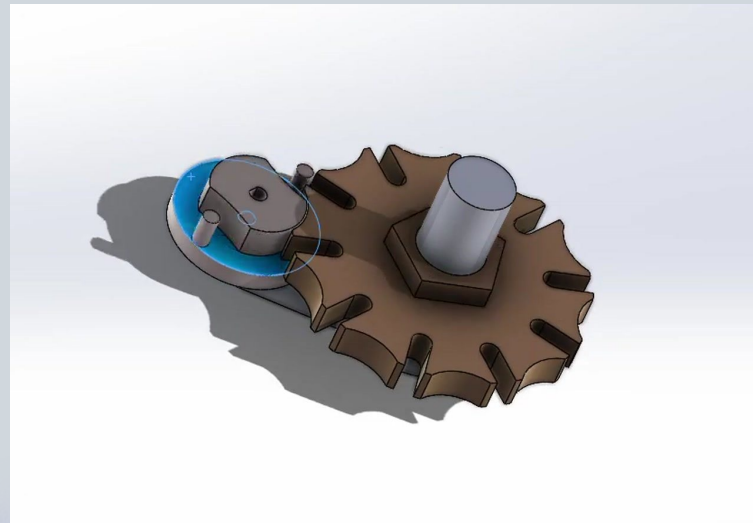
Paint-Mixer



Paint-Mixer



Hardware



Geneva Mechanism

The Geneva mechanism ensures reliable and repeatable rotary indexing, guaranteeing the device consistently rotates to and locks in its predetermined positions.

Magnetic clips

Locked into position magnetically, it allows for low-effort access to paint cartridges when replacing

Vibration motor

Allows for the final product to be well emulsified using low frequency vibrations

Iterations

Prototype 2



Based on company requirements for improved usability and manufacturability, the design was revised to a rectangular plastic housing with magnetic doors. Internally, it retains the same string claw mechanism, linear actuator, stepper motors, and gear assemblies for automated colour mixing.

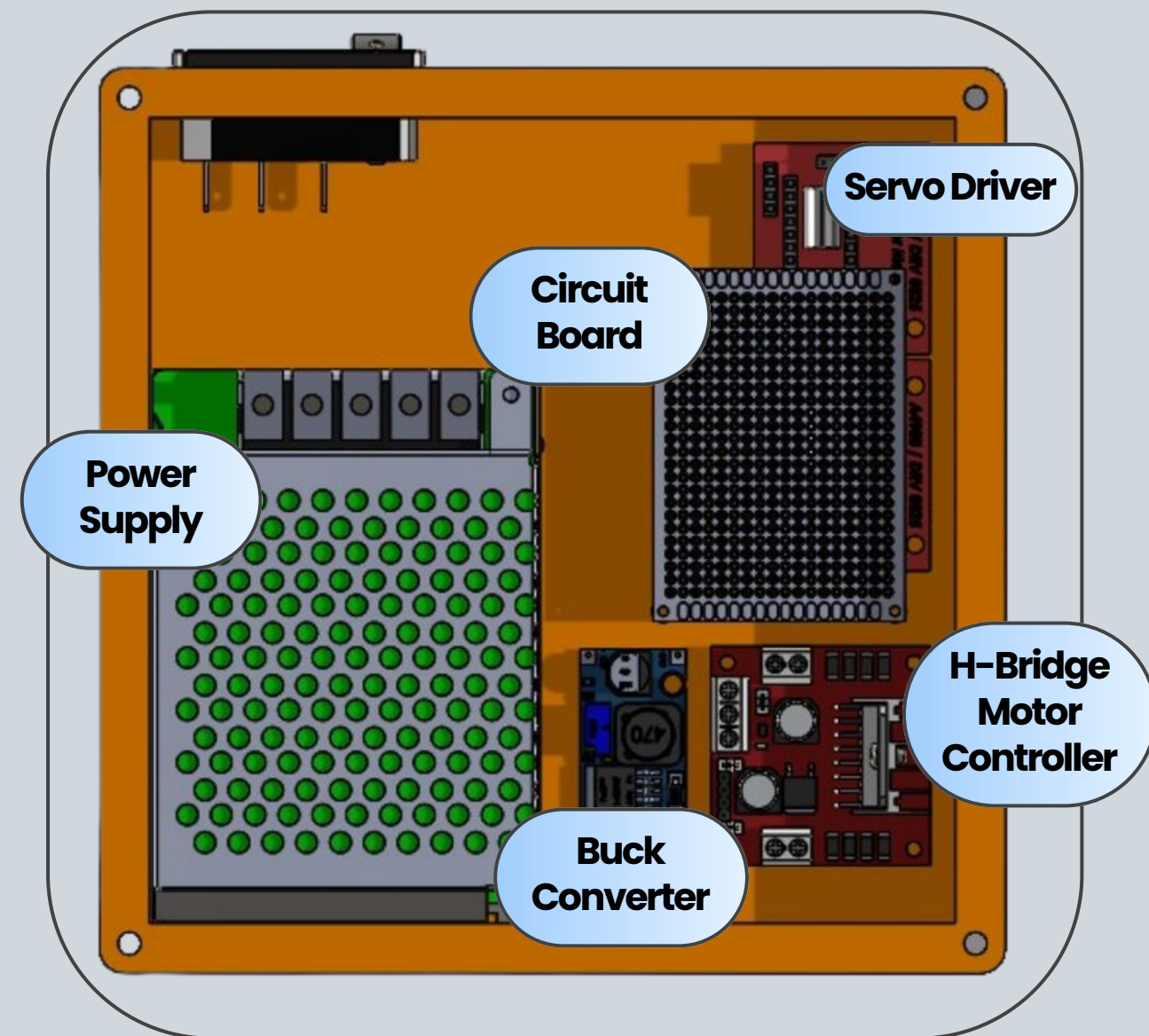
Final

Things to consider

- Manufacturability
- The price of manufacturing
- Quality of user interaction
- Explore other redundancies

Electronics

The ESP32-C3 serves as the central controller, coordinating all operations including stepper motor movement, actuator control, and user interface display.



Linear Actuator

The linear actuator is powered directly from the 24V supply and controlled via the ESP32 using a relay or motor driver interface, allowing controlled vertical motion for mixing or dispensing operations.

Stepper Motors

A NEMA 17 stepper motor, driven by an A4988 stepper motor driver, precisely controls the syringes for colour dispensing. The driver allows microstepping, enabling fine-grained motion control for accurate fluid measurement.

Screen

User interaction is provided through a 3.5" ILI9488 TFT LCD touch screen, connected to the ESP32 via SPI, which displays menus, colour inputs, and real-time status.

Software

```
#include <Arduino.h>

// On board LED pin
const int ON_LED = 8;

// Stepper motor driver A4988 pins
const int StepperMotorPins[2][2] = {
  {7, 8},
  {9, 10}
};

const int steps_per_rev = 200;

// Linear actuator L298N motor driver pins
const int LIN_OUT = 20;
const int LIN_IN = 21;

// stepper motor directions (for function "stepper_motor()")
const int ANTICLOCKWISE = 1;
const int CLOCKWISE = 0;
// linear actuator directions (for function "linear_actuator()")
const int OUT = 1;
const int IN = 0;

// flashes onboard LED
void flash_led(int number = 3){
  Serial.println("flash_led - number =");
  Serial.println(number);

  for (int i = 0; i <= number; i++){
    digitalWrite(ON_LED, HIGH);
    delay(100);
    digitalWrite(ON_LED, LOW);
    delay(100);
  }
}

// direction = 0 (clockwise), direction = 1 (anti-clockwise)
// - the direction is in relation to the stepper motor faced up, remember to convert directions on gears
void stepper_motor(int chosen_motor, int direction = 0, int revolutions = 1, int speed = 1500){

  int DIR = StepperMotorPins[chosen_motor][0];
  int STEP = StepperMotorPins[chosen_motor][1];
```

ESP32 working code

```
#include <Arduino.h>
#include <cmath>
#include <cstring>

// ===== Configuration =====
constexpr float TOTAL_ML = 10.0f; // (kept for reference, not used in ratio output)

// Pigment order: C, M, Y, K, W
const float PIGMENTS[5][3] = {
  { 0.0f, 255.0f, 255.0f }, // C
  {255.0f, 0.0f, 255.0f }, // M
  {255.0f, 255.0f, 0.0f }, // Y
  { 0.0f, 0.0f, 0.0f }, // K
  {255.0f, 255.0f, 255.0f } // W
};

bool hexToRgb(const char* hex, float rgb[3]) {
  if (hex[0] == '#') hex++;
  if (strlen(hex) != 6) return false;
  for (int i = 0; i < 3; i++) {
    char byte[3] = {hex[2*i], hex[2*i+1], '\0'};
    rgb[i] = (float)strtol(byte, nullptr, 16);
  }
  return true;
}

void solveLeastSquares(const float target[3], float x[5]) {
  float MTM[5][5] = {0};
  float MTb[5] = {0};

  for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
      for (int k = 0; k < 3; k++) {
        MTM[i][j] += PIGMENTS[i][k] * PIGMENTS[j][k];
      }
    }
    for (int k = 0; k < 3; k++) {
      MTb[i] += PIGMENTS[i][k] * target[k];
    }
  }

  // 📌 New: Convert fractions to integer percentages that sum to 100
  void fractionsToRoundedPercentages(const float fractions[5], int percentages[5]) {
    float scaled[5];
    int rounded[5];
    int sum = 0;

    // Step 1: Scale to 100 and round down
    for (int i = 0; i < 5; i++) {
      scaled[i] = fractions[i] * 100.0f;
      rounded[i] = (int)floorf(scaled[i]);
      sum += rounded[i];
    }

    // Copy to output
    for (int i = 0; i < 5; i++) {
      percentages[i] = rounded[i];
    }
  }

  // 📌 Main function: returns [C, M, Y, K, W] as integer percentages summing to 100
  bool computeCMYKWPercentages(const char* hexColor, int ratios[5]) {
    float target[3];
    if (!hexToRgb(hexColor, target)) {
      return false;
    }
  }
```

Hex code to volume code

Software

```
#include <Arduino.h>

// On board LED pin
const int ON_LED = 8;

// Stepper motor driver A4988 pins
const int StepperMotorPins[2][2] = {
  {7, 8},
  {9, 10}
};
const int steps_per_rev = 200;

// Linear actuator L298N motor driver pins
const int LIN_OUT = 20;
const int LIN_IN = 21;

// stepper motor directions (for function "stepper_motor()")
const int ANTICLOCKWISE = 1;
const int CLOCKWISE = 0;
// linear actuator directions (for function "linear_actuator()")
const int OUT = 1;
const int IN = 0;

// flashes onboard LED
void flash_led(int number = 3){
  Serial.println("flash_led - number =");
  Serial.println(number);

  for (int i = 0; i <= number; i++){
    digitalWrite(ON_LED, HIGH);
    delay(100);
    digitalWrite(ON_LED, LOW);
    delay(100);
  }
}

// direction = 0 (clockwise), direction = 1 (anti-clockwise)
// - the direction is in relation to the stepper motor faced up, remember to convert directions on gears
void stepper_motor(int chosen_motor, int direction = 0, int revolutions = 1, int speed = 1500){

  int DIR = StepperMotorPins[chosen_motor][0];
  int STEP = StepperMotorPins[chosen_motor][1];
```

ESP32 working code

- Controls:
 - Motors to change the colour selection
 - Linear Actuators for precise dispensing of paint volumes
 - Hosting the WebUI for user colour selection

Software

Hex code to volume code

- **Analyses** the user selected colour
- **Calculates** the paint volumes required for the selected Hex Code
- **Sends the data** to the motors for accurate colour mixing

Ink Mixer





Default: 50 mL

Select a colour and click "Confirm Colour"

Color

Basic colors:

Custom colors:

Define Custom Colors >>

 Hue: 88 Red: 68
Sat: 152 Green: 213
Lum: 132 Blue: 97

```
#include <Arduino.h>
#include <cmath>
#include <cstring>

// ===== Configuration =====
constexpr float TOTAL_ML = 10.0f; // (kept for reference, not used in ratio output)

// Pigment order: C, M, Y, K, W
const float PIGMENTS[5][3] = {
  { 0.0f, 255.0f, 255.0f }, // C
  {255.0f, 0.0f, 255.0f }, // M
  {255.0f, 255.0f, 0.0f }, // Y
  { 0.0f, 0.0f, 0.0f }, // K
  {255.0f, 255.0f, 255.0f } // W
};

bool hexToRgb(const char* hex, float rgb[3]) {
  if (hex[0] == '#') hex++;
  if (strlen(hex) != 6) return false;
  for (int i = 0; i < 3; i++) {
    char byte[3] = {hex[2*i], hex[2*i+1], '\0'};
    rgb[i] = (float)strtol(byte, nullptr, 16);
  }
}
```

```
es(const float target[3], float x[5]) {
  = {0};
  0);
  l < 5; i++) {
    0; j < 5; j++) {
      k = 0; k < 3; k++) {
        l[j] += PIGMENTS[i][k] * PIGMENTS[j][k];
      }
    }
  }
  0; k < 3; k++) {
    = PIGMENTS[i][k] * target[k];
  }
  actions to integer percentages that sum to 100
  ledPercentages(const float fractions[5], int percentages[5]) {
```

```
to 100 and round down
l < 5; i++) {
  fractions[i] * 100.0f;
  (int)floorf(scaled[i]);
  ed[i];
}
```

```
l < 5; i++) {
  l = rounded[i];
}
```

```
returns [C, M, Y, K, W] as integer percentages summing to 100
entages(const char* hexColor, int ratios[5]) {
```

```
(Color, target)) {
```

Image To Model



Image To Model

Moxin Clients:



Image To Model

Efficient AI Assisted Design Tool



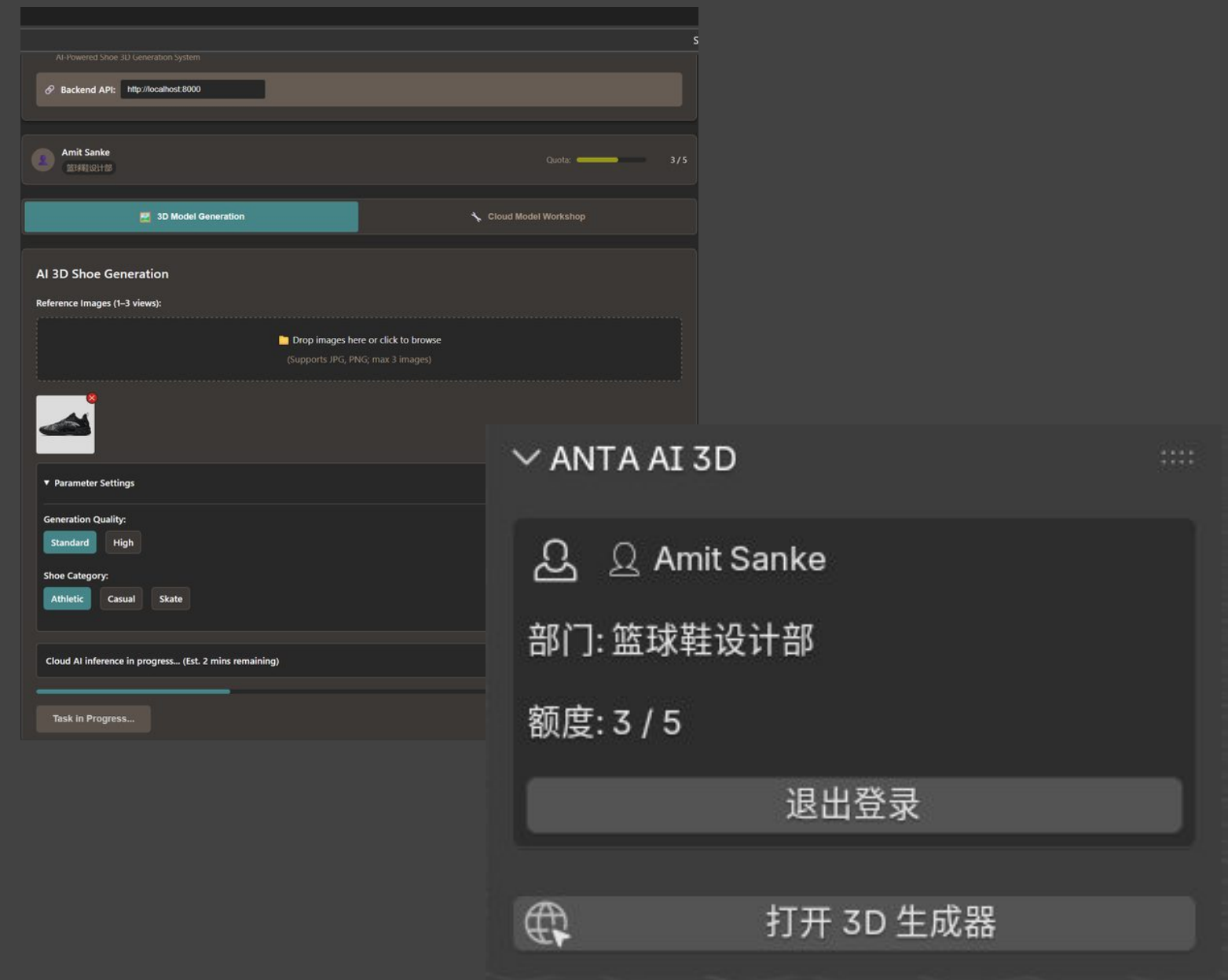
Backend

```
File Edit Selection View Go Run ... ImageTo3DModel
cleaner.py main.py model.py 1 X requirements.txt
back&gt; hunyuan > model.py > HunyuanModel > generate_obj_from_image
25 class HunyuanModel:
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Starting image to 3D model conversion pipeline
=====
[1/2] Converting image to 3D model...
      Input: assets/demo.png
Loading Hunyuan3D-2 pipeline on device: cuda
2025-12-08 14:54:09,277 - hy3dgen.shapgen - INFO - Try to load model from local path: /root/.cache/hy3dgen/tencent/Hunyuan3D-2/hunyuan3d-dit-v2-0
2025-12-08 14:54:09,284 - hy3dgen.shapgen - INFO - Loading model from /root/.cache/hy3dgen/tencent/Hunyuan3D-2/hunyuan3d-dit-v2-0/model.fp16.safetensors
PointCrossAttentionEncoder INFO: pc_sharpedge_size is given, using pc_size=5120, pc_sharpedge_size=5120
✓ Loaded Hunyuan3D-2 from local cache.
Loading image: assets/demo.png
Running Hunyuan3D-2 inference...
Diffusion Sampling:: 100%|██████████| 50/50 [00:09:00:00, 5.26it/s]
Volume Decoding: 100%|██████████| 7134/7134 [00:18:00:00, 391.92it/s]
OBJ written: assets/demo_raw.obj
      Output: assets/demo_raw.obj

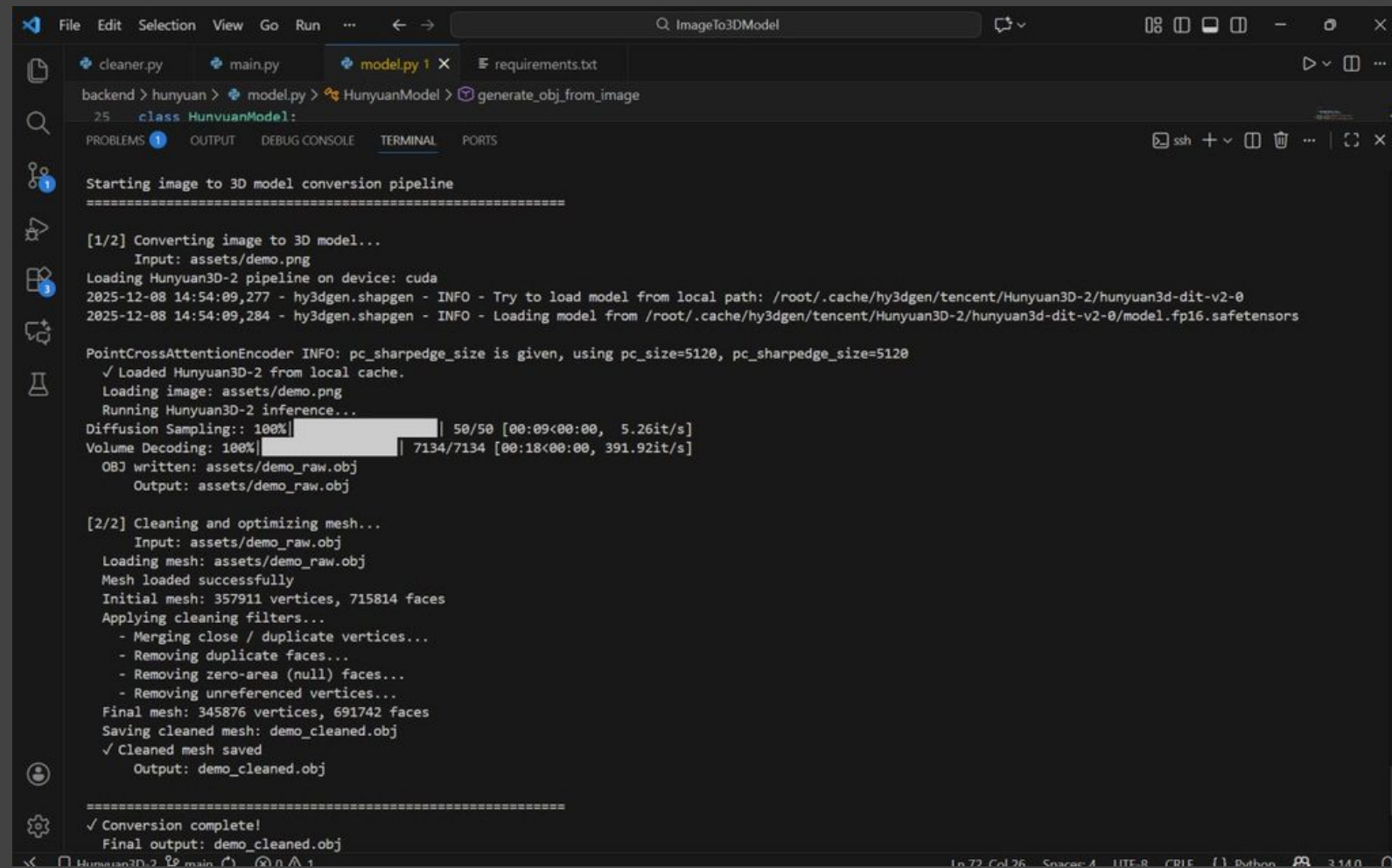
[2/2] Cleaning and optimizing mesh...
      Input: assets/demo_raw.obj
Loading mesh: assets/demo_raw.obj
Mesh loaded successfully
Initial mesh: 357911 vertices, 715814 faces
Applying cleaning filters...
- Merging close / duplicate vertices...
- Removing duplicate faces...
- Removing zero-area (null) faces...
- Removing unreferenced vertices...
Final mesh: 345876 vertices, 691742 faces
Saving cleaned mesh: demo_cleaned.obj
✓ Cleaned mesh saved
      Output: demo_cleaned.obj

=====
✓ Conversion complete!
Final output: demo_cleaned.obj
```

Frontend



Backend



```
File Edit Selection View Go Run ... ImageTo3DModel
cleaner.py main.py model.py 1 X requirements.txt
backend > hunyuan > model.py > HunyuanModel > generate_obj_from_image
25 class HunyuanModel:
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Starting image to 3D model conversion pipeline
=====
[1/2] Converting image to 3D model...
Input: assets/demo.png
Loading Hunyuan3D-2 pipeline on device: cuda
2025-12-08 14:54:09,277 - hy3dgen.shapgen - INFO - Try to load model from local path: /root/.cache/hy3dgen/tencent/Hunyuan3D-2/hunyuan3d-dit-v2-0
2025-12-08 14:54:09,284 - hy3dgen.shapgen - INFO - Loading model from /root/.cache/hy3dgen/tencent/Hunyuan3D-2/hunyuan3d-dit-v2-0/model.fp16.safetensors
PointCrossAttentionEncoder INFO: pc_sharpedge_size is given, using pc_size=5120, pc_sharpedge_size=5120
✓ Loaded Hunyuan3D-2 from local cache.
Loading image: assets/demo.png
Running Hunyuan3D-2 inference...
Diffusion Sampling:: 100%|██████████| 50/50 [00:09:00:00, 5.26it/s]
Volume Decoding: 100%|██████████| 7134/7134 [00:18:00:00, 391.92it/s]
OBJ written: assets/demo_raw.obj
Output: assets/demo_raw.obj

[2/2] Cleaning and optimizing mesh...
Input: assets/demo_raw.obj
Loading mesh: assets/demo_raw.obj
Mesh loaded successfully
Initial mesh: 357911 vertices, 715814 faces
Applying cleaning filters...
- Merging close / duplicate vertices...
- Removing duplicate faces...
- Removing zero-area (null) faces...
- Removing unreferenced vertices...
Final mesh: 345876 vertices, 691742 faces
Saving cleaned mesh: demo_cleaned.obj
✓ Cleaned mesh saved
Output: demo_cleaned.obj

=====
✓ Conversion complete!
Final output: demo_cleaned.obj
```

3D Model Generation

- Importing AI Model Weights
- Mesh Post-processing (Pymeshlab)

Server Configuration

- Set up Nvidia A100 GPUs

Frontend

AI 3D Generation

Server Address:

http://192.168.1.100:8000

Section 1: Model Generation (Hunyuan 3D)

Select Image

No images selected

Generate Model from Image

Status: Idle

Section 2: Mesh ...sing (Pymeshlab)

Apply to Selected Model:

Model Simplification

Surface Smoothing

Topological Uniformization

One - Click Denoising

Replace Original Model After Proces...

Execute Post-processing

Status: Idle

Hunyuan 3D Image-to-Mesh Generator

Backend Server URL: http://localhost:8000

Generate 3D Model

Process Mesh

Image to 3D Model Generation

Select Images: Choose Files No file chosen

Generate 3D Model

AI-Powered SHOE 3D Generation System

Backend API: http://localhost:8000

Amit Sanke

Quota: 3 / 5

3D Model Generation

Cloud Model Workshop

AI 3D Shoe Generation

Reference Images (1-3 views):

Drop images here or click to browse
(Supports JPG, PNG, max 3 images)



Parameter Settings

Generation Quality: Standard High

Shoe Category: Athletic Casual Skate

Cloud AI inference in progress... (Est. 2 mins remaining)

Task in Progress...

ANTA AI 3D

Amit Sanke

部门: 篮球鞋设计部

额度: 3 / 5

退出登录

打开 3D 生成器

Blender Plugin



End Results

Image to Model Blender Plugin



End Results



Image to Model Blender Plugin



Experience and Takeaways

This internship provided hands-on exposure to the complete product development cycle, from problem identification to hardware and software implementation. Working in a multidisciplinary team strengthened our technical skills, collaboration, and real-world engineering mindset.

- Our Contribution to the company : We designed and developed functional hardware and software prototypes that addressed real product needs and contributed directly to ongoing development efforts.
- Our growth and gains with company's support : With continuous guidance and feedback, we improved our technical depth, problem-solving ability, and understanding of industry-level design and execution.



Experience and Takeaways



Completion status of our project :

The **first functional version of the system has been completed**, with core features implemented and tested.

Future development plans :

The software project will be extended into a membership-based platform for image-to-Blender conversion, with plans to commercialize it through custom 3D-printed memorial and keepsake products.



Thank You

for your time
and attention